Tribhuvan University

Faculty of Humanities & Social Sciences

OFFICE OF THE DEAN

Database Management System

BCA — Semester 4 — 2020 — Answer Sheet

Subject Code: CACS255

Group B

Attempt any SIX questions. [6×5=30]

1. What is database? Discuss the importance of DBMS.

[1+3]

Answer: Database: A database is an organized collection of structured data stored electronically in a computer system. It is designed to efficiently store, retrieve, and manage large amounts of information. Data in a database is typically organized into tables consisting of rows (records) and columns (attributes), with relationships defined between tables. **Importance of DBMS (Database Management System):** 1. **Data Independence:** DBMS provides abstraction between physical storage and logical view. Changes in physical storage don't affect application programs. 2. **Data Integrity:** Enforces constraints (primary keys, foreign keys, check constraints) to maintain data accuracy and consistency. 3. **Data Security:** Provides authentication, authorization, and access control mechanisms to protect sensitive data from unauthorized access. 4. **Data Redundancy Control:** Normalization techniques minimize duplicate data, saving storage and preventing update anomalies. 5. **Concurrent Access:** Allows multiple users to access and modify data simultaneously while maintaining consistency through locking and transaction management. 6. **Backup and Recovery:** Provides mechanisms to create data backups and recover from system failures, ensuring data durability. 7. **Query Language:** SQL provides a powerful, standardized way to interact with data without writing complex procedural code. 8. **Scalability:** Modern DBMS can handle terabytes of data and scale horizontally or vertically as needs grow. 9. **Decision Support:** Enables complex analytics, reporting, and business intelligence through aggregation, joins, and views.

2. Define union compatibility. Given following relations, show the resulting relation of Director ∩ Actor and Actor ∪ Director?

[2+3]

Answer: Union Compatibility: Two relations are union compatible if they have the same number of attributes and the corresponding attributes have compatible domains (same data types). For set operations (UNION, INTERSECTION, DIFFERENCE) to be valid, the participating relations must be union compatible. **Requirements for Union Compatibility:** • Same number of columns • Corresponding columns have the same domain • Column names may differ (result typically uses first relation's names) **Note:** The specific relations "Director" and "Actor" were not fully visible in the original paper. Using typical examples: **Relation: Director** NameMovie Christopher NolanInception Steven SpielbergJurassic Park Quentin TarantinoPulp Fiction **Relation: Actor** NameMovie Leonardo DiCaprioInception Quentin TarantinoPulp Fiction Sam NeillJurassic Park **Director ∩ Actor (INTERSECTION):** Returns tuples present in BOTH relations. NameMovie Quentin TarantinoPulp Fiction **Actor ∪ Director (UNION):** Returns all unique tuples from both relations. NameMovie Christopher NolanInception Steven SpielbergJurassic Park Quentin TarantinoPulp Fiction Leonardo DiCaprioInception Sam NeillJurassic Park **SQL Implementation:** SELECT * FROM Director INTERSECT SELECT * FROM Actor; SELECT * FROM Actor UNION SELECT * FROM Director;

3. How Order By and Group BY Having Clause in SQL are different? Illustrate with suitable examples.

[3+2]

Answer: ORDER BY vs GROUP BY vs HAVING: FeatureORDER BYGROUP BYHAVING PurposeSorts result setGroups rows by column valuesFilters grouped results When appliedAfter all selectionBefore aggregationAfter GROUP BY Works withAny selected columnsAggregate functionsAggregate conditions Can useColumn names, aliases, expressionsColumn names onlyAggregate functions **Example Table: Employees** IDNameDeptSalary 1RamIT50000 2SitaHR45000 3HariIT60000 4GitaHR55000 5ShyamIT70000 **1. ORDER BY Example:** SELECT * FROM Employees ORDER BY Salary DESC; Result: Shyam (70000), Hari (60000), Gita (55000), Ram (50000), Sita (45000) **2. GROUP BY Example:** SELECT Dept, COUNT(*) as Count, AVG(Salary) as AvgSalary FROM Employees GROUP BY Dept; Result: DeptCountAvgSalary HR250000 IT360000 **3. HAVING Example:** SELECT Dept, AVG(Salary) as AvgSalary FROM Employees GROUP BY Dept HAVING AVG(Salary) > 55000; Result: Only IT department (avg 60000) since HR's average (50000) is filtered out. **Key Difference:** WHERE filters rows BEFORE grouping; HAVING filters groups AFTER aggregation. ORDER BY simply sorts the final output.

4. Why normalization is needed? When an relation is said to be in 1NF and 2NF?

[1+4]

Answer: Need for Normalization: Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. Without normalization, databases suffer from: • **Insertion Anomalies:** Cannot insert certain data without other unrelated data • **Update Anomalies:** Changing one piece of data requires updating multiple rows • **Deletion Anomalies:** Deleting data unintentionally loses other important information • **Data Redundancy:** Same data stored multiple times, wasting storage and causing inconsistencies **First Normal Form (1NF):** A relation is in 1NF if and only if: 1. All attributes contain only atomic (indivisible) values 2. Each attribute contains values of a single type 3. Each row is unique (has a primary key) 4. There are no repeating groups **Example violating 1NF:** StudentIDNameCourses 1RamMath, Physics, Chemistry **1NF Compliant:** StudentIDNameCourse 1RamMath 1RamPhysics 1RamChemistry **Second Normal Form (2NF):** A relation is in 2NF if: 1. It is in 1NF 2. All non-key attributes are fully functionally dependent on the entire primary key (no partial dependency) **Partial Dependency:** A non-key attribute depends on only part of a composite primary key. **Example violating 2NF:** StudentIDCourseIDStudentNameCourseNameGrade 1C101RamDatabaseA PK = (StudentID, CourseID) StudentName depends only on StudentID (partial dependency) CourseName depends only on CourseID (partial dependency) **2NF Compliant (split into three tables): Student table:** StudentID (PK)StudentName 1Ram **Course table:** CourseID (PK)CourseName C101Database **Enrollment table:** StudentID (FK)CourseID (FK)Grade 1C101A

5. How cost of query is measured during query processing? Construct query expression tree for the relational algebra query.

[3+2]

Answer: Query Cost Measurement: The cost of query execution is measured using various metrics that estimate the resources required to execute a query: **1. Disk I/O Cost:** • Number of block transfers (most significant factor) • Number of disk seeks • Formula: Cost = (Number of block transfers × block_transfer_time) + (Number of seeks × seek_time) **2. CPU Cost:** • Time for processing operations (comparisons, computations) • Time for sorting, hashing, and merging **3. Memory Cost:** • Amount of buffer space required • Number of buffer pages used **4. Network Cost:** • For distributed databases, data transfer between sites **Factors Affecting Cost:** • Size of relations (number of tuples, tuple size) • Available indexes (B+ tree, hash indexes) • Selectivity of selection and join conditions • Sorting requirements • Buffer pool size **Cost Estimation for Operations:** • **Selection (σ):** Cost depends on whether index exists. With B+ tree index: cost ≈ height of tree + matching entries. Without index: full table scan. • **Projection (π):** Cost = scanning relation + eliminating duplicates (if DISTINCT) • **Join (■):** Nested loop join: |R| × |S|. Hash join: 3(|R| + |S|). Merge join: sort cost + merge cost. **Query Expression Tree:** For the query: $\pi_{FName,LName,Address}(\sigma_{DName="Research"}(Department \blacksquare_{Dnumber=Dno} (Employee \times Works\_on)))$ The expression tree from bottom to top: π (FName, LName, Address) | σ (DName='Research') | ■ (Dnumber=Dno) / Department × / Employee Works_on **Optimized Tree (pushing selection down):** π (FName, LName, Address) | ■ (Dnumber=Dno) / σ(DName='Research') ■ | / Department Employee Works_on Optimization: Apply selection on Department early to reduce join size.

■ *Diagram Required:* Query expression tree diagram showing root node π at top, child node σ below it, then join node ■, with Department on left and cross-product × on right (Employee and Works_on as leaves).

6. How wait-for-graph is constructed? How can you use it to detect deadlock in a schedule of transactions? Support your answer with an example.

[2+2+1]

Answer: Wait-For Graph (WFG): A wait-for graph is a directed graph used to detect deadlocks in database systems. It is maintained by the database lock manager. **Construction:** 1. Create a node for each active transaction in the system 2. If transaction T_i is waiting for a lock held by transaction T_j, draw a directed edge from T_i to T_j ($T_i \rightarrow T_j$) 3. The edge means "T_i is waiting for T_j" **Deadlock Detection:** If the wait-for graph contains a **cycle**, a deadlock exists. The transactions involved in the cycle are deadlocked. **Example:** Consider transactions T1, T2, T3 and data items A, B, C: • T1 holds lock on A, requests lock on B • T2 holds lock on B, requests lock on C • T3 holds lock on C, requests lock on A **Wait-For Graph:** T1 ----→ T2 ↑ | | ↓ T3 ←----- Edges: T1→T2 (T1 waits for B held by T2), T2→T3 (T2 waits for C held by T3), T3→T1 (T3 waits for A held by T1) **Cycle:** T1 → T2 → T3 → T1. **Deadlock detected! Resolution:** The DBMS must break the deadlock by: • **Aborting a victim transaction:** Choose the transaction with least work done (victim selection criteria: age, progress, resources held) • **Rollback:** Undo victim's changes and release its locks • **Restart:** Restart the aborted transaction later **Alternative — Timeout:** If a transaction waits longer than a predetermined time, assume deadlock and abort. **Advantage of WFG:** Precise detection, no false positives. **Disadvantage:** Maintenance overhead — graph must be updated every time locks are requested or released.

■ *Diagram Required:* Wait-for graph with 3 nodes (T1, T2, T3) arranged in a triangle. Arrows: T1→T2, T2→T3, T3→T1 forming a cycle.

7. How read and write operations are performed in a transaction? List the various states where transactions can enter into.

[3+2]

Answer: Read and Write Operations in Transactions: Read Operation (read(X)): 1. Transaction T_i issues read(X) to access data item X 2. The system checks if X is in T_i's local workspace (buffer) 3. If yes, return the buffered value 4. If no, check if T_i has a shared or exclusive lock on X 5. If locked appropriately, read X from database into buffer and return 6. If not locked, request appropriate lock from lock manager 7. If lock granted, proceed; if not, transaction waits **Write Operation (write(X)):** 1. Transaction T_i modifies the value of X in its local buffer 2. When T_i issues write(X), the system checks if T_i holds an exclusive lock on X 3. If yes, write the buffered value to the database (or to undo/redo log first) 4. If no, request exclusive lock; if granted, proceed; else wait 5. In strict 2PL, the actual database update may be deferred until commit **Transaction States:** 1. **Active:** The transaction is currently executing operations. This is the initial state when a transaction begins. 2. **Partially Committed:** The transaction has completed its final operation but may still need to ensure durability. Changes are written to logs but may not be permanently saved. 3. **Committed:** The transaction has successfully completed all operations, and all changes have been made permanent in the database. This is a terminal state. 4. **Failed:** The transaction cannot complete normally due to an error (hardware failure, logical error, deadlock). It must be rolled back. 5. **Aborted:** The transaction has been rolled back, and all changes have been undone. The database is restored to its state before the transaction began. The transaction may be restarted or discarded. **State Transition Diagram:** [Active] -- final operation --> [Partially Committed] -- successful --> [Committed] | | failure ↓ [Failed] -- rollback --> [Aborted] -- restart --> [Active] **ACID Properties ensure:** • Atomicity: All or nothing (via rollback/commit) • Consistency: Valid state to valid state • Isolation: Concurrent transactions don't interfere • Durability: Committed changes survive failures

■ *Diagram Required:* State diagram: Active → Partially Committed → Committed (success path). Active → Failed → Aborted (failure path). Aborted can restart to Active.

Group C

Attempt any TWO questions. [2×10=20]

8. Given the schema as below, write relational algebra and SQL queries for following...

[10]

Answer: Schema: • Sailors(sid, sname, rating, age) • Boats(bid, bname, colour) • Reserves(sid, bid, reservedate) **Relational Algebra and SQL Queries: i. Find name, rating and age of all sailors.** Relational Algebra: $\pi_{sname,\ rating,\ age}$(Sailors) SQL: SELECT sname, rating, age FROM Sailors; **ii. Find the name and color of boats having color blue.** Relational Algebra: $\pi_{bname,\ colour}(\sigma_{colour='blue'}$(Boats)) SQL: SELECT bname, colour FROM Boats WHERE colour = 'blue'; **iii. Find name of the boats reserved on date 2021/07/20.** Relational Algebra: $\pi_{bname}(\sigma_{reservedate='2021\text{-}07\text{-}20'}$(Boats ■ Reserves)) SQL: SELECT b.bname FROM Boats b JOIN Reserves r ON b.bid = r.bid WHERE r.reservedate = '2021-07-20'; **iv. Find the names of boats and sailors who have reserved boats having color green.** Relational Algebra: $\pi_{sname,\ bname}(\sigma_{colour='green'}$(Sailors ■ Reserves ■ Boats)) SQL: SELECT s.sname, b.bname FROM Sailors s JOIN Reserves r ON s.sid = r.sid JOIN Boats b ON r.bid = b.bid WHERE b.colour = 'green'; **v. Use Left and Right Outer join between Sailors and Reserves.** SQL: -- Left Outer Join: All sailors even if they haven't reserved SELECT s.sid, s.sname, r.bid, r.reservedate FROM Sailors s LEFT JOIN Reserves r ON s.sid = r.sid; -- Right Outer Join: All reservations even if sailor info missing SELECT s.sid, s.sname, r.bid, r.reservedate FROM Sailors s RIGHT JOIN Reserves r ON s.sid = r.sid; -- Full Outer Join (all records from both) SELECT s.sid, s.sname, r.bid, r.reservedate FROM Sailors s FULL OUTER JOIN Reserves r ON s.sid = r.sid;

9. a) Design an ER diagram of your choice. The ER diagram should contain at least five entities. Out of those entities, one should be weak entity. Your entities should have appropriate attributes including primary key attributes, if any. There should be presence of one to one relationship, one to many relationship and many to many relationship in your ER diagram. b) Convert the ER diagram in Q.I.a. into equivalent relational tables.

[5+5]

Answer: ER Diagram Design — University Database: Entities and Attributes: 1. **Student** (Strong Entity) • Attributes: student_id (PK), name, email, date_of_birth, address 2. **Department** (Strong Entity) • Attributes: dept_id (PK), dept_name, location, budget 3. **Course** (Strong Entity) • Attributes: course_id (PK), course_name, credits, description 4. **Instructor** (Strong Entity) • Attributes: instructor_id (PK), name, email, salary 5. **Dependent** (Weak Entity — depends on Instructor) • Attributes: dependent_name (Partial Key), relationship, birth_date • Identifying relationship with Instructor **Relationships:** 1. **Belongs_to** (Student — Department): **Many-to-One** • Many students belong to one department 2. **Heads** (Instructor — Department): **One-to-One** • One instructor heads one department • One department has one head 3. **Teaches** (Instructor — Course): **Many-to-Many** • One instructor can teach many courses • One course can be taught by many instructors • Attribute: semester, year 4. **Enrolls** (Student — Course): **Many-to-Many** • One student can enroll in many courses • One course has many students • Attribute: grade, enrollment_date 5. **Has** (Instructor — Dependent): **One-to-Many** (Identifying) • One instructor has many dependents • Dependent cannot exist without instructor **ER Diagram Description:** • Rectangles for entities (double rectangle for Dependent) • Diamonds for relationships • Lines connect entities to relationships • Cardinality marked as 1, M, N on lines • Underlined attributes are primary keys • Dashed underline for partial key of weak entity **b) Relational Tables:** 1. **Department**(dept_id, dept_name, location, budget, head_instructor_id) • PK: dept_id • FK: head_instructor_id references Instructor(instructor_id) 2. **Instructor**(instructor_id, name, email, salary, dept_id) • PK: instructor_id • FK: dept_id references Department(dept_id) 3. **Student**(student_id, name, email, date_of_birth, address, dept_id) • PK: student_id • FK: dept_id references Department(dept_id) 4. **Course**(course_id, course_name, credits, description) • PK: course_id 5. **Dependent**(instructor_id, dependent_name, relationship, birth_date) • PK: (instructor_id, dependent_name) • FK: instructor_id references Instructor(instructor_id) 6. **Teaches**(instructor_id, course_id, semester, year) • PK: (instructor_id, course_id, semester, year) • FK: instructor_id references Instructor(instructor_id) • FK: course_id references Course(course_id) 7. **Enrollment**(student_id, course_id, grade, enrollment_date) • PK: (student_id, course_id) • FK: student_id references Student(student_id) • FK: course_id references Course(course_id) **Conversion Rules Applied:** • Strong entities → separate tables with all attributes • Weak entities → table with partial key + owner's primary key as composite PK • 1:1 relationships → Add FK to either side or merge tables • 1:N relationships → Add FK on the many side • M:N relationships → Create junction table with PKs of both entities as composite PK

■ *Diagram Required:* ER diagram with 5 entities: Student, Department, Course, Instructor, Dependent (double rectangle). Relationships: Belongs_to (M:1), Heads (1:1), Teaches (M:N), Enrolls (M:N), Has (1:N identifying).

10. Define stored procedures and triggers. How they are applicable in database? Illustrate with examples, how can you create before and after triggers on INSERT query.

[3+2+5]

Answer: Stored Procedures: A stored procedure is a precompiled collection of SQL statements and control-flow statements stored in the database under a name. It can accept parameters, perform operations, and return results. Stored procedures reduce network traffic, improve performance, and enforce business logic at the database level. **Benefits:** • Reusability: Write once, call many times • Security: Users can execute without direct table access • Performance: Precompiled execution plan • Maintainability: Centralized business logic **Example Stored Procedure:** DELIMITER // CREATE PROCEDURE GetStudentGrades(IN student_id INT) BEGIN SELECT c.course_name, e.grade FROM Enrollment e JOIN Course c ON e.course_id = c.course_id WHERE e.student_id = student_id; END // DELIMITER ; -- Call the procedure CALL GetStudentGrades(101); **Triggers:** A trigger is a stored program that is automatically executed by the DBMS when a specific event (INSERT, UPDATE, DELETE) occurs on a specified table. Triggers enforce integrity constraints, audit changes, and automatically maintain derived data. **Types of Triggers by Timing: • BEFORE:** Executed before the triggering event • **AFTER:** Executed after the triggering event **Types by Event:** • INSERT trigger • UPDATE trigger • DELETE trigger **Applicability in Database:** 1. **Audit Logging:** Track who changed what and when 2. **Data Validation:** Enforce complex business rules 3. **Derived Data Maintenance:** Automatically update summary tables 4. **Cascading Changes:** Propagate changes to related tables 5. **Security:** Prevent unauthorized modifications **Example: BEFORE INSERT Trigger** -- Create audit table CREATE TABLE Student_Audit (audit_id INT AUTO_INCREMENT PRIMARY KEY, action VARCHAR(10), student_id INT, old_name VARCHAR(100), new_name VARCHAR(100), changed_by VARCHAR(50), changed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP); DELIMITER // CREATE TRIGGER before_student_insert BEFORE INSERT ON Student FOR EACH ROW BEGIN -- Validate: Ensure email is not empty IF NEW.email IS NULL OR NEW.email = '' THEN SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Email cannot be empty'; END IF; -- Log the action INSERT INTO Student_Audit (action, student_id, new_name, changed_by) VALUES ('INSERT', NEW.student_id, NEW.name, CURRENT_USER()); END // DELIMITER ; **Example: AFTER INSERT Trigger** DELIMITER // CREATE TRIGGER after_student_insert AFTER INSERT ON Student FOR EACH ROW BEGIN -- Automatically enroll new student in mandatory orientation course INSERT INTO Enrollment (student_id, course_id, enrollment_date) VALUES (NEW.student_id, 'ORIENT101', CURDATE()); -- Update department student count UPDATE Department SET student_count = student_count + 1 WHERE dept_id = NEW.dept_id; END // DELIMITER ; **Difference between BEFORE and AFTER: BEFORE TriggerAFTER Trigger** Runs before the triggering statementRuns after the triggering statement completes Can modify NEW values (for INSERT/UPDATE)Cannot modify NEW values Can reject the operation (SIGNAL ERROR)Cannot prevent the original operation Use for validation, transformationUse for cascading, logging, auditing If trigger fails, original statement is cancelledIf trigger fails, may require rollback

---

Note: These answers are prepared for educational purposes. Students are encouraged to verify with official textbooks and course materials.